

pŷ# Skyward Airlines Pro - Complete Project Completion Blueprint

Version: 1.0
Project root: `E:\airline-management-system-final-project`
Backend: `backend/airline` - Spring Boot REST API
Frontend: `frontend/aircrat-management1` - Angular 13 SPA

1. Purpose

This document is a complete, module-wise completion plan for your existing airline management and reservation project. It is written for both humans and AI assistants: each section names the current state, the missing work, the recommended backend/frontend files, the API connection points, and acceptance criteria. The goal is to finish the system while preserving your current folder structure, coding style, UI direction, and frontend/backend connection pattern.

2. Current Project Snapshot

```
E:\airline-management-system-final-project
% % %  backend/airline
%      % % %  pom.xml
%      % % %  src/main/java/com/cogent
%          % % %  airline
%          % % %  config
%          % % %  controller
%          % % %  dao
%          % % %  dto
%          % % %  model
%          % % %  repository
%          % % %  security
%          % % %  service
% % %  frontend/aircrat-management1
% % %  src/app
% % %  core
%      % % %  guards
%      % % %  interceptors
%      % % %  models
%      % % %  services
%      % % %  utils
% % %  features
% % %  layout
% % %  shared
```

Observed source size: backend has 60 Java source files; frontend app has 137 TypeScript, 58 HTML, and 58 CSS files.

Backend stack: Java 17, Spring Boot 4.0.5, Spring Web, Spring Data JPA/Hibernate, MySQL, Spring Security, JWT via JJWT 0.12.6, OpenPDF, Maven wrapper.

Frontend stack: Angular 13.0.3, TypeScript 4.4.4, RxJS, Angular Router lazy loading, Forms/Reactive Forms, Chart.js/ng2-charts, jsPDF/html2canvas.

3. Architecture Understanding

Backend currently follows this layered shape:

Controller -> Service -> DAO/Repository -> Model/DTO -> MySQL
Security/Config support JWT, CORS, and stateless API behavior.

This should remain the core backend style. Some older modules use DAO classes and newer modules use Spring Data repositories. For this project, it is acceptable to keep that hybrid style, but all business rules should live in service classes and controllers should stay thin.

Frontend currently follows this shape:

App routing -> lazy feature modules -> feature components -> core services -> backend API
Shared UI components provide sidebar/navbar/header/footer.

The UI is an admin airline operations dashboard. The sidebar already groups modules into Operations, Booking & Pax, Finance, Analytics, and System. Preserve that design direction.

4. Existing Backend API Surface

Module	Controller	Base Path	Status
Aircraft	AircraftController	/api/aircrafts	Exists
Airline	AirlineController	/api/airlines	Exists
Booking	BookingController	/api/bookings	Exists
Dashboard	DashboardController	/api/dashboard	Exists
Expense	ExpenseController	/api/expenses	Exists
Flight	FlightController	/api/flights	Exists
Tracking	FlightTrackingController	/tracking	Exists, but path should be standardized
Loyalty	LoyaltyController	/api/loyalty	Exists
Passenger	PassengerController	/api/passengers	Exists
Payment	PaymentController	/api/payments	Exists
Refund	RefundController	/api/refunds	Exists

Frontend modules that currently need missing backend support: auth, users, pricing, coupon, revenue, waitlist/overbooking.

5. Critical Findings From The Codebase

1. `AuthGuard` returns `true`; it does not protect routes yet.
2. `RoleGuard` returns `true`; role-based routing is not active yet.
3. `JwtInterceptor` forwards requests without attaching the JWT token.
4. Backend `SecurityConfig` currently permits all requests; this is okay during development but not for final delivery.
5. Frontend services hard-code `http://localhost:8080`; these should use `environment.apiUrl`.
6. Tracking backend uses `/tracking`, while most modules use `/api/...`; standardize to `/api/tracking` or intentionally align the frontend service.
7. In `BookingService.createBooking`, this appears wrong: when `paymentStatus` is null, code calls `booking.setStatus("PAID")`. It should set payment status, and a professional flow should usually start as `UNPAID` or `PENDING\_PAYMENT`.
8. Reports are partly routed through `/dashboard/...` even though a report module exists. Keep dashboard for operational KPIs and reports for report pages.
9. Several backend comments show encoding artifacts. Clean gradually without changing behavior.
10. Important statuses are strings; convert to enums carefully.
11. There is no global backend error response format yet.
12. `ddl-auto=update` is useful for development but database migrations are better for final delivery.
13. JWT secret and database credentials are in `application.properties`; use environment variables for production.

6. Roadmap By Priority

Phase 1 - Foundation

- Move API base URLs into Angular environments.
- Implement real AuthGuard, RoleGuard, and JwtInterceptor.
- Add backend auth/user model, controller, service, repository.
- Add role-based backend security after login works.
- Add global exception handler with consistent error JSON.
- Add DTO request/response classes for create/update workflows.
- Add validation annotations and Angular form validation.
- Standardize tracking API path.

Phase 2 - Missing Modules

- User Management backend.
- Pricing Rules backend.
- Coupon backend.
- Waitlist and Overbooking backend.
- Revenue aggregation backend.
- Report routing cleanup and backend report endpoints.

Phase 3 - Reservation Workflow

- Search flights by origin, destination, departure date, class, passenger count.
- Quote fare from pricing rules, taxes, coupon, and loyalty.
- Hold and select seats safely.
- Create PNR booking as `PENDING\_PAYMENT`.

- Confirm booking only after successful payment.
- Generate ticket/invoice only for confirmed booking.
- Award loyalty points after payment.
- Process cancellation/refund with policy and approval.
- Update reports, revenue, load factor, and flight tracking.

Phase 4 - Production Hardening

- Add unit, controller, repository, and integration tests.
- Add Flyway or Liquibase migrations.
- Add OpenAPI/Swagger documentation.
- Add request logging and audit fields.
- Move secrets out of committed properties.
- Add exportable reports and final README instructions.

7. Module-Wise Completion Plan

7.1 Authentication And User Management

Current state: frontend has login service, login screen, guards, and user-management screens. Backend has JWT utility/filter/security config, but no visible `AuthController`, `User` entity, or user management API.

Recommended backend files:

```
model/User.java
model/Role.java
dto/AuthDTO.java
repository/UserRepository.java
service/AuthService.java
service/UserService.java
controller/AuthController.java
controller/UserController.java
```

Recommended API:

Method	Path	Purpose
POST	/auth/login	Validate credentials and return JWT/user profile
POST	/auth/change-password	Change password
GET	/api/users	List users
GET	/api/users/{id}	User detail
POST	/api/users	Create user
PUT	/api/users/{id}	Update user
PATCH	/api/users/{id}/toggle-active	Enable/disable user
DELETE	/api/users/{id}	Soft delete user

Frontend tasks: implement guards, attach JWT in interceptor, use real current user in sidebar, redirect expired sessions to `/auth/login`, hide menu items by role.

Recommended roles: `SUPER\_ADMIN`, `ADMIN`, `MANAGER`, `AGENT`, `OPS`, `VIEWER`.

Acceptance criteria: login works end-to-end, protected routes are blocked without token, backend enforces roles, and user screens are connected to real API data.

7.2 Aircraft Management

Current state: backend/frontend CRUD exists at `/api/aircrafts`.

Tasks:

- Validate registration/tail number, model, manufacturer, capacity, and status.
- Add unique registration number.
- Add statuses: `ACTIVE`, `MAINTENANCE`, `RETIRED`, `GROUNDED`.
- Link aircraft to flights.
- Prevent scheduling grounded aircraft.
- Add maintenance due date and last maintenance date.
- Add frontend filters and status badges.
- Protect delete when assigned to future flights.

Acceptance criteria: aircraft can be managed safely and used in scheduling rules.

7.3 Airline Management

Current state: backend/frontend CRUD exists at `/api/airlines`.

Tasks:

- Validate airline code and name.
- Enforce unique airline code/name.
- Add country, support email, phone, website, and active status.
- Connect airline to aircraft and flights.
- Prevent delete when active records depend on it.

Acceptance criteria: airlines are clean master data and can be selected in flight operations.

7.4 Flight Management

Current state: backend has `Flight`, `FlightController`, `FlightService`, and `FlightDAO`. Flight creation calculates distance, arrival time, and class prices from base price.

Strengths already present:

- Flight number duplicate check.
- Airport distance calculation.
- Arrival date/time derived from estimated speed.
- Economy/Premium/Business/First Class prices derived from base price.

Tasks:

- Use airport code as canonical key instead of city-only matching.
- Add aircraft assignment.
- Prevent aircraft schedule conflicts.
- Convert status string to enum.
- Add pagination and filtering.
- Validate arrival after departure.
- Add timezone support for international routes.
- Add status values: `SCHEDULED`, `DELAYED`, `BOARDING`, `DEPARTED`, `ARRIVED`, `CANCELLED`.

Recommended API additions:

Method	Path	Purpose
GET	/api/flights/search	Reservation search
GET	/api/flights/airports	Airport choices
PATCH	/api/flights/{id}/status	Update flight status

Acceptance criteria: flights can be searched, booked, tracked, and priced consistently.

7.5 Passenger Management

Current state: backend/frontend CRUD exists at `/api/passengers`.

Tasks:

- Validate passport, email, phone, date of birth, nationality.
- Add passenger type: `ADULT`, `CHILD`, `INFANT`.
- Link passenger with loyalty account.
- Add duplicate detection by passport/email.
- Add travel history from bookings.
- Use active/inactive instead of hard delete when booking history exists.

Acceptance criteria: passenger profile supports booking, loyalty, payment, refund, and reporting.

7.6 Booking And Reservation Management

Current state: booking has PNR generation, passenger/flight snapshots, seat map, flight seat report, and search. Frontend has booking list/detail, seat selection, ticket view, invoice, and tracking.

Important bug to fix:

```
if (booking.getPaymentStatus() == null) booking.setStatus("PAID");
```

Recommended correction:

```
if (booking.getPaymentStatus() == null) booking.setPaymentStatus("UNPAID");
if (booking.getStatus() == null) booking.setStatus("PENDING_PAYMENT");
```

Recommended booking statuses: `PENDING\_PAYMENT`, `CONFIRMED`, `CHECKED\_IN`, `CANCELLED`, `REFUNDED`, `NO\_SHOW`.

Recommended payment statuses: `UNPAID`, `PAID`, `FAILED`, `REFUND\_PENDING`, `REFUNDED`, `PARTIALLY\_REFUNDED`.

Tasks:

- Make booking creation transactional.
- Prevent duplicate seats for the same flight.
- Validate seat class against selected fare class.
- Add unpaid booking expiration.
- Add cancellation endpoint.
- Generate ticket only for confirmed booking.
- Add quote endpoint before booking.
- Add baggage, fare class, passenger count, and special service fields if needed.

Acceptance criteria: no double booking, clear booking lifecycle, and payment/refund modules update booking state correctly.

7.7 Seat Selection

Current state: backend generates 119 seats: First rows 1-2, Business rows 3-5, Premium rows 6-8, Economy rows 9-20.

Tasks:

- Store aircraft seat layout instead of hardcoding one layout globally.
- Add seat statuses: `AVAILABLE`, `HELD`, `BOOKED`, `BLOCKED`.
- Add temporary hold expiration.
- Validate class-zone matching.
- Allow admin/ops to block seats.

Acceptance criteria: seat map is accurate and cannot be double-booked under concurrent requests.

7.8 Payment Management

Current state: backend has payment endpoints and stats; frontend has list/detail/process views.

Tasks:

- Link payment to valid booking.
- Payment success updates booking to `CONFIRMED` and payment to `PAID`.
- Payment failure keeps booking pending and payment failed.
- Add transaction reference uniqueness.
- Add idempotency key to prevent duplicate charges.
- Support card, mobile banking, cash, bank transfer, loyalty points, mixed payment.
- Add gateway abstraction for future real gateway integration.

Acceptance criteria: payment cannot succeed for missing/cancelled booking and cannot be duplicated accidentally.

7.9 Refund Management

Current state: backend/frontend exist with list, status, booking lookup, preview, initiate, approve, process, and reject.

Tasks:

- Add refund policy by fare class and time before departure.
- Prevent refund if booking is unpaid or already refunded.
- Add workflow: agent initiates, manager approves, finance processes.
- Refund processing updates booking and payment status.
- Reverse loyalty points when needed.
- Add reason codes and audit notes.

Acceptance criteria: refund amount is explainable, policy-based, auditable, and role-controlled.

7.10 Coupon And Promotion Management

Current state: frontend exists; backend source is missing.

Recommended backend files:

```
model/Coupon.java
model/CouponRedemption.java
dto/CouponDTO.java
repository/CouponRepository.java
repository/CouponRedemptionRepository.java
service/CouponService.java
controller/CouponController.java
```

Fields: code, name, description, discount type, value, max discount, start/end date, minimum booking amount, applicable class/route, usage limit, usage count, per-passenger limit, active.

Recommended API:

Method	Path	Purpose
GET	/api/coupons	List coupons
POST	/api/coupons	Create coupon
PUT	/api/coupons/{id}	Update coupon
DELETE	/api/coupons/{id}	Disable coupon
POST	/api/coupons/validate	Validate coupon for quote
POST	/api/coupons/redeem	Redeem after booking/payment

Acceptance criteria: coupon validates before booking and cannot exceed usage rules.

7.11 Pricing And Fare Rules

Current state: frontend exists; backend source is missing. Current flight pricing uses fixed multipliers.

Recommended backend files:

```
model/PricingRule.java
dto/PricingDTO.java
repository/PricingRuleRepository.java
service/PricingService.java
controller/PricingController.java
```

Rule types: route-based price, class multiplier, seasonal multiplier, load factor, advance purchase discount, weekend/holiday surcharge, last-minute surcharge, loyalty discount, coupon interaction.

Recommended API:

Method	Path	Purpose
GET	/api/pricing-rules	List rules
POST	/api/pricing-rules	Create rule
PUT	/api/pricing-rules/{id}	Update rule
DELETE	/api/pricing-rules/{id}	Disable rule
POST	/api/pricing/simulate	Simulate price
POST	/api/pricing/quote	Final quote

Acceptance criteria: booking price is calculated by a dedicated pricing service and can be simulated before publishing.

7.12 Loyalty Management

Current state: backend/frontend exist. Backend includes account, transaction, enrollment, award, redeem, bonus, tiers, stats, top earners, and transaction history.

Tasks:

- Award points automatically after successful payment.
- Reverse points after refund/cancellation.
- Add tier calculation based on points or flight count.
- Add point expiration.
- Add redemption limits and fraud checks.
- Enforce member number uniqueness.
- Link loyalty to passenger profile and booking history.

Recommended tiers: `BRONZE`, `SILVER`, `GOLD`, `PLATINUM`.

Acceptance criteria: loyalty is connected to booking, payment, and refund lifecycle.

7.13 Waitlist And Overbooking

Current state: frontend exists; backend source is missing.

Recommended backend files:

```
model/WaitlistEntry.java
model/OverbookingRule.java
dto/WaitlistDTO.java
repository/WaitlistRepository.java
service/WaitlistService.java
controller/WaitlistController.java
```

Workflow:

1. Passenger requests seat on full flight.
2. System adds passenger to waitlist by priority.
3. If seat opens, system promotes next eligible passenger.
4. Passenger has limited time to confirm/pay.
5. Overbooking rules allow controlled oversell based on route/history.

Priority factors: loyalty tier, fare class, booking time, passenger type, disruption/reaccommodation case.

Recommended API:

Method	Path	Purpose
GET	/api/waitlist	List queue
POST	/api/waitlist	Add passenger
PATCH	/api/waitlist/{id}/promote	Promote passenger
PATCH	/api/waitlist/{id}/cancel	Cancel queue item
GET	/api/waitlist/flight/{flightId}	Queue by flight
GET	/api/overbooking/rules	List rules
POST	/api/overbooking/rules	Create/update rule

Acceptance criteria: waitlist can convert to booking and overbooking is controlled by authorized roles.

7.14 Flight Tracking And Operations

Current state: backend exists at /tracking; frontend has tracking list/status/update.

Tasks:

- Standardize base path to /api/tracking.
- Track status changes with timestamp, remarks, location, gate, terminal, delay reason.
- Add status history timeline.
- Add latest public/status lookup by flight number.
- Restrict updates to operations roles.
- Feed delayed/cancelled/arrived counts into dashboard.

Recommended statuses: `SCHEDULED`, `CHECK\_IN\_OPEN`, `BOARDING`, `DEPARTED`, `IN\_AIR`, `LANDED`, `ARRIVED`, `DELAYED`, `CANCELLED`, `DIVERTED`.

Acceptance criteria: tracking shows current and historical status and every update is auditable.

7.15 Expense Management

Current state: backend/frontend exist with CRUD and total expense endpoint.

Tasks:

- Add expense categories as controlled values or separate entity.
- Add receipt/invoice attachment support if required.
- Add approval status: draft, submitted, approved, rejected, paid.
- Add cost center: flight, aircraft, route, department.
- Add monthly and category summaries.
- Validate amount/date/category.
- Prevent hard delete after approval.

Acceptance criteria: expenses feed profit/loss and financial reports accurately.

7.16 Revenue And Financial Analytics

Current state: frontend exists. Backend has dashboard sales/expense chart endpoints but no dedicated revenue controller.

Recommended backend files:

```
dto/RevenueDTO.java
service/RevenueService.java
controller/RevenueController.java
```

Recommended API:

Method	Path	Purpose
GET	/api/revenue/kpis	Revenue, expense, profit, refund totals
GET	/api/revenue/profit-loss	P/L by period
GET	/api/revenue/forecast	Forecast
GET	/api/revenue/route-performance	Revenue/load factor by route
GET	/api/revenue/class-mix	Revenue by fare class

Acceptance criteria: revenue uses real paid bookings, subtracts refunds and expenses, and supports dashboard/report charts.

7.17 Dashboard And Reports

Current state: dashboard has summary, sales chart, and expense chart endpoints. Frontend has dashboard and report components.

Tasks:

- Keep `/dashboard` for operational KPIs.
- Use `/report` for formal reports.
- Add date-range filters.
- Add export to PDF/CSV.
- Add backend report endpoints instead of only frontend calculations.
- Add loading, empty, and error states for charts.

Suggested reports: sales, expenses, profit/loss, flight seat/load factor, route performance, refund, loyalty liability, payment method.

Acceptance criteria: reports match backend data, can be filtered, and can be exported.

8. Cross-Cutting Standards

Backend Standards

- Use DTOs for external API requests/responses.
- Use enums for statuses and controlled values.
- Add validation annotations: `@NotNull`, `@NotBlank`, `@Email`, `@Positive`.
- Add `@RestControllerAdvice` for consistent errors.
- Keep business rules in services.
- Keep transaction boundaries in service methods.
- Add pagination for list endpoints.
- Add audit columns for important entities.
- Move secrets to environment variables.
- Add OpenAPI documentation.

Frontend Standards

- Use `environment.apiUrl`.
- Keep one Angular service per backend domain.
- Match TypeScript interfaces with backend DTOs.
- Add loading, empty, error, and success states.
- Use reactive forms for complex forms.
- Enforce route-level permissions and sidebar visibility by role.
- Keep UI consistent with the current admin dashboard style.
- Add shared status badge, confirm dialog, toast, and data table components.
- Move duplicated business logic into services/utilities.

Database Standards

- Add Flyway or Liquibase migrations.
- Index flight number, booking reference, passenger email, passport, payment reference.

- Add foreign keys where possible.
- Avoid hard deleting transactional records.
- Use locking/versioning for seat, booking, and payment-critical flows.

Security Standards

- Use BCrypt for passwords.
- Enforce backend role rules, not only frontend guards.
- Disable destructive operations for low-privilege users.
- Move JWT secret and DB credentials out of committed properties.
- Configure CORS per environment.
- Validate every request body server-side.

Testing Standards

Backend tests:

- Service tests for pricing, booking, payment, refund, loyalty.
- Controller tests for API contracts and security.
- DAO/repository tests for query correctness.
- Integration test for booking -> payment -> ticket -> refund.

Frontend tests:

- Service tests for API URL correctness.
- Guard tests for auth/role routing.
- Component tests for forms and validation.
- E2E smoke tests for login, search, booking, payment, ticket, refund.

9. Target End-To-End Airline Reservation Flow

1. Admin creates airline and aircraft.
2. Ops manager creates flights with route, date, time, aircraft, base fare.
3. Pricing manager configures dynamic pricing rules.
4. Agent searches flights for passenger.
5. Passenger profile is selected or created.
6. System quotes fare by route, class, seat, coupon, taxes, loyalty.
7. Seat is selected and temporarily held.
8. Booking is created with PNR and PENDING_PAYMENT status.
9. Payment is processed.
10. Booking becomes CONFIRMED and ticket/invoice become available.
11. Loyalty points are awarded.
12. Flight status updates appear in tracking.
13. If cancelled/refunded, refund policy is calculated, approved, processed.
14. Reports update revenue, refunds, expenses, and load factor.

10. Recommended Sprint Order

Sprint 1 - Real Auth And API Configuration

- Backend auth/user model/controller/service.
- Frontend guards/interceptor.
- Environment-based API URL.
- Sidebar user and role visibility.

Sprint 2 - Booking Core Reliability

- Fix payment status bug.
- Add booking statuses and validation.
- Add seat duplicate protection.
- Connect payment to booking confirmation.

Sprint 3 - Missing Finance Modules

- Coupon backend.
- Pricing backend.

- Revenue backend.
- Align frontend services to endpoints.

Sprint 4 - Operations Modules

- Waitlist backend.
- Overbooking rules.
- Tracking path standardization and status history.
- Aircraft-flight assignment rules.

Sprint 5 - Reports, Testing, And Polish

- Report route cleanup.
- Exportable reports.
- Error handling.
- Unit/integration tests.
- README and deployment notes.

11. Suggested Folder Additions

```
backend/airline/src/main/java/com/cogent/exception
backend/airline/src/main/java/com/cogent/common
backend/airline/src/main/java/com/cogent/dto/*Request.java
backend/airline/src/main/java/com/cogent/dto/*Response.java
backend/airline/src/main/resources/db/migration
frontend/aircrat-management1/src/app/shared/components/confirm-dialog
frontend/aircrat-management1/src/app/shared/components/status-badge
frontend/aircrat-management1/src/app/shared/components/data-table
frontend/aircrat-management1/src/app/shared/components/toast
```

Do not move existing feature modules unless a refactor is planned and tested.

12. Definition Of Done

The project is complete when:

1. All frontend services call real backend endpoints.
2. No important feature screen depends on mock data.
3. Login, roles, JWT, and route protection work end-to-end.
4. Admin can manage airlines, aircraft, users, flights, pricing, and coupons.
5. Agent can create passenger, search flight, select seat, book, process payment, and view ticket.
6. Finance/manager can approve refunds and view revenue reports.
7. Ops can update tracking and monitor flight status.
8. Waitlist and overbooking are connected to flight capacity.
9. Reports are filterable and exportable.
10. Backend validates data and returns consistent errors.
11. Tests cover reservation, payment, refund, pricing, and security flows.
12. README explains setup and running the full system.
13. Database migrations define schema reproducibly.
14. The app runs locally with backend on 8080 and frontend on 4200.

13. AI-Readable Checklist

project:

```
name: Skyward Airlines Pro
backend: Spring Boot Java 17 MySQL
frontend: Angular 13 TypeScript
preserve_structure: true
priority_order:
  - auth_security
  - api_url_environment_config
  - booking_payment_integrity
  - missing_backend_modules
  - reporting_and_exports
  - tests_and_documentation
missing_backend_modules:
```

- auth
- users
- pricing
- coupons
- waitlist
- revenue

critical_fixes:

- implement_auth_guard
- implement_role_guard
- implement_jwt_interceptor
- remove_backend_permit_all_after_auth
- fix_booking_payment_status_assignment
- standardize_tracking_api_path
- move_api_urls_to_environment
- add_global_exception_handler
- replace_status_strings_with_enums
- add_validation_and_dtos

core_workflow:

- airline_aircraft_setup
- flight_scheduling
- pricing_quote
- passenger_profile
- seat_selection
- booking_pnr
- payment_confirmation
- ticket_invoice
- loyalty_award
- tracking_update
- refund_if_needed
- reports

14. Closing Recommendation

Your project already has a strong feature-module skeleton and many important backend domains. The next work is integration discipline: complete missing backend modules, make auth real, centralize API configuration, enforce booking/payment/refund rules transactionally, and standardize routes, statuses, validation, and errors. Follow the sprint order above to turn the current broad prototype into a coherent professional airline reservation and operations system.